SISTEMA DE COMPRAS E CONTROLE DE ESTOQUE

Churrascaria Medieval

DOCUMENTAÇÃO FUNCIONAL E TÉCNICA

Documento atualizado a partir do código, das configurações e dos arquivos de modelo de dados presentes no repositório do projeto.

1. Apresentação

O projeto propõe um sistema web para apoiar o controle de compras e estoque da Churrascaria Medieval. A aplicação centraliza cadastros de produtos, grupos, unidades de medida e fornecedores; mantém registros de estoque; organiza cotações e pedidos; e oferece operações para registrar saídas e movimentações.

Objetivos do sistema

- Reduzir retrabalho com listas e planilhas separadas.
- Manter dados de compras, fornecedores e estoque em uma aplicação integrada.
- Registrar cotações, ofertas e pedidos com seus itens e estados.
- Apoiar o acompanhamento de entradas, saídas e ajustes de estoque.

Contexto operacional

O processo informado pela equipe organiza as compras por categoria e calendário: bebidas são contadas no domingo para chegada prevista na terça-feira; linha seca, na terça-feira para chegada na quarta-feira; e carnes, na quinta-feira para chegada na sexta-feira. O sistema deve apoiar essa rotina sem eliminar a conferência física realizada pela equipe.

2. Fluxo de negócio

Fluxo de referência

Etapa	Atividade
1. Cadastro	Manter grupos, unidades de medida, produtos, usuários e fornecedores.
2. Controle	Consultar o saldo e registrar entradas, saídas ou ajustes de estoque.
3. Cotação	Abrir uma cotação, incluir produtos e registrar as ofertas recebidas dos fornecedores.
4. Decisão	Comparar as ofertas e selecionar as condições de compra.
5. Pedido	Criar o pedido associado ao fornecedor e aos itens escolhidos.
6. Recebimento	Conferir o que chegou e atualizar o estado do pedido e o estoque.
7. Histórico	Consultar os registros de estoque e compras disponíveis no sistema.

Categoria	Dia de contagem informado	Chegada prevista informada
Bebidas	Domingo	Terça-feira
Linha seca	Terça-feira	Quarta-feira
Carnes	Quinta-feira	Sexta-feira

Rotina de contagem por categoria

O repositório inclui tela de estoque com consulta e pesquisa e operações para cadastrar produto com saldo inicial e ajustar quantidade.

3. Funcionalidades presentes no repositório

Área	O que o código já contempla
Acesso e usuários	Tela de login ligada ao endpoint de autenticação; cadastro, consulta, edição e remoção de usuários; rotas de interface com controle de acesso no cliente.
Catálogo	Cadastro e manutenção de grupos de produtos, unidades de medida e produtos. Produto pode ser associado a grupo e unidade.
Fornecedores	Cadastro e consulta de fornecedores; a listagem apresenta contato e atalho para WhatsApp.
Estoque	Consulta e pesquisa de saldos; cadastro de estoque associado a produto; atualização de quantidade; endpoints de movimentação e de saídas.
Cotações	Cadastro de cotação, itens associados e registro de oferta de fornecedor para item cotado.
Pedidos	Cadastro e consulta de pedidos e itens; status de pedido inclui rascunho, enviado, parcialmente recebido, recebido e cancelado. A interface tem fluxo de confirmação de pedido que atualiza o saldo de estoque.
Administração	Componentes de gestão de usuários e páginas de navegação para as áreas do sistema.

Pontos previstos ou não confirmados como fluxo completo

- Listas digitais recorrentes geradas automaticamente conforme o calendário de compras.
- Registro de contagem física com histórico próprio, data e responsável vinculados a cada lista.
- Sugestão automática de compra pela diferença entre saldo e estoque mínimo.
- Conferência completa da nota fiscal com registro estruturado de itens faltantes, trocados e adicionais e seus encaminhamentos.
- Acompanhamento integral das ocorrências até sua resolução, incluindo devolução, reenvio ou entrega emergencial.
- Painel de auditoria acessível pela interface. O modelo de auditoria existe no contexto de dados, mas não foi localizada uma tela ou API específica para consultá-lo.

4. Arquitetura da aplicação

A solução está separada em três partes: interface web, API e persistência de dados.

Camada	Tecnologias e responsabilidades
Frontend	React com TypeScript, Vite e React Router. Páginas e componentes ficam em frontend/src. A comunicação com a API usa fetch e a variável VITE_API_URL.
Backend	ASP.NET Core com C# e .NET 10. Controllers recebem as requisições HTTP; services aplicam regras e consultam o banco; DTOs definem os dados de entrada e resposta.
Persistência	Entity Framework Core. A configuração atual usa PostgreSQL e a connection string de desenvolvimento aponta para o arquivo app.db. Existem migrations para criar e alterar o esquema.
Documentação da API	OpenAPI e Scalar são configurados para o ambiente de desenvolvimento.
Tratamento de falhas	Middleware centralizado e exceções de domínio, negócio, conflito e não encontrado.

Fluxo técnico de uma operação

Interface React → endpoint REST do controller → serviço de domínio → AppDbContext / Entity Framework Core → banco PostgreSQL.

O projeto também contém Medieval DB.sql em formato de modelo DBML com PostgreSQL indicado como tipo de banco. Essa modelagem é uma referência conceitual; a configuração executável presente no backend usa PostgreSQL . Para uma apresentação, é importante explicar essa diferença.

Autenticação

O backend disponibiliza login por e-mail e senha e verifica a senha usando BCrypt. No frontend, o estado de sessão é armazenado no localStorage e usado pelos componentes de rota para controlar a navegação. O código atual não configura middleware de autorização no pipeline da API; portanto, as rotas protegidas da interface representam uma proteção de navegação no cliente.

5. Organização dos dados

O AppDbContext reúne os conjuntos de dados abaixo. As entidades se relacionam por identificadores e as configurações do Entity Framework definem nomes de tabela, chaves e regras.

Entidade	Finalidade
Usuario	Conta, grupo de acesso, estado e dados de login.
GrupoProduto e UnidadeMedida	Classificação do catálogo e unidade usada para quantidades.
Produto	Identificação do item, grupo, unidade, estoque mínimo/máximo, custos e validade.
Fornecedor	Dados comerciais, contato, endereço, dias de entrega e condição de pagamento.
Cotacao, CotacaoItem e CotacaoItemFornecedor	Cabeçalho da cotação, produtos/quantidades solicitados e ofertas recebidas por fornecedor.
PedidoCompra e PedidoCompraItem	Pedido ao fornecedor, usuário responsável, datas, estado, totais e itens/quantidades.
Estoque	Saldo disponível e valor total associados a um produto.
MovimentacaoEstoque	Histórico de variações, quantidade anterior/posterior, origem e referência relacionada.
SaidaEstoque e SaidaEstoqueItem	Registro de saídas por consumo, perda, vencimento, devolução, ajuste ou outro motivo.
Auditoria	Estrutura para registrar operações e dados anteriores/novos.

Relacionamentos principais

- Um grupo classifica vários produtos; cada produto usa uma unidade de medida.
- Cada produto possui um registro de estoque no modelo da aplicação.
- Uma cotação tem itens; cada item pode receber ofertas de vários fornecedores.
- Uma cotação pode estar relacionada a pedidos de compra; cada pedido é associado a um fornecedor e possui itens.
- Saídas e movimentações relacionam produto, usuário e estoque conforme o tipo de operação.

Estados de negócio relevantes

Registro	Estados encontrados no código
Cotação	Rascunho, enviada, em resposta, encerrada e cancelada.

Pedido de compra	Rascunho, enviado, parcialmente recebido, recebido e cancelado.
Saída de estoque	Rascunho, confirmada e cancelada.
Usuário	Ativo e inativo.

6. API e recursos

Os controllers seguem o padrão REST e, em geral, oferecem consulta de coleção, consulta por identificador, criação, atualização e exclusão. Os recursos principais são:

Recurso	Rota base
Usuários e login	/api/Usuarios | /api/Usuarios/login
Grupos de produtos	/api/GruposProdutos
Unidades de medida	/api/UnidadesMedida
Produtos	/api/Produtos
Fornecedores	/api/Fornecedores
Estoque	/api/Estoques
Cotações	/api/Cotacoes
Itens da cotação	/api/CotacaoItens
Ofertas de fornecedores	/api/CotacaoItemFornecedores
Pedidos e itens	/api/PedidosCompra | /api/PedidosCompraItens
Saídas e itens	/api/SaidasEstoque | /api/SaidasEstoqueItens
Movimentações de estoque	/api/MovimentacoesEstoque

As rotas de desenvolvimento da API são definidas no launchSettings: HTTP em http://localhost:5283 e HTTPS em https://localhost:7196. O frontend e a API precisam estar ativos ao mesmo tempo para que as telas que consomem dados funcionem.

7. Como executar o projeto

Pré-requisitos

- Node.js e npm para instalar e executar o frontend.
- .NET SDK 10 para compilar e executar a API.

1. Preparar e iniciar o backend

Na pasta raiz do repositório, execute:

```
dotnet ef database update --project backend/MedievalApi
dotnet run --project backend/MedievalApi
```

A primeira linha aplica as migrations existentes ao banco PostgreSQL de desenvolvimento, caso a ferramenta dotnet-ef esteja instalada. A API é configurada para usar http://localhost:5283 no perfil HTTP.

2. Configurar e iniciar o frontend

Crie o arquivo frontend/.env.local com o endereço da API e depois instale as dependências e inicie o Vite:

```
VITE_API_URL=http://localhost:5283

cd frontend
npm install
npm run dev
```

Abra http://localhost:5173 no navegador. O valor de VITE_API_URL é usado tanto no login quanto nas chamadas de dados. A política CORS atual da API permite localhost:5173 e 127.0.0.1:5173.

Estrutura principal do repositório

Pasta/arquivo	Conteúdo
frontend/src/pages	Páginas da interface: login, home, estoque, fornecedores, cotações, pedidos, saídas, usuários e cadastros auxiliares.
frontend/src/components	Layout, listas, modais, seletores, pesquisas e cartões usados pelas páginas.
frontend/services/api.ts	Função comum para chamadas HTTP da interface.
backend/MedievalApi/Controllers	Endpoints HTTP da API.
backend/MedievalApi/Services	Regras e operações de negócio.
backend/MedievalApi/Models e DTOs	Entidades e contratos de entrada/resposta.
backend/MedievalApi/Data	Contexto, configurações de entidades e migrations.
Medieval DB.sql	Modelo conceitual do banco em sintaxe DBML.
docs/	Fichas de acompanhamento e documentos do projeto.

8. Resultados esperados e próximos passos

Benefícios esperados

- Dados de fornecedores, produtos, compras e estoque organizados em um só sistema.
- Menos redigitação entre papel, planilha e pedido.
- Rastreabilidade de alterações e movimentações conforme os recursos disponíveis.
- Melhor visualização das cotações e dos estados dos pedidos.
- Base para reduzir compras urgentes e evitar falta ou excesso de produtos.

Evoluções recomendadas

Prioridade	Evolução
1	Implementar listas de contagem por categoria e dia, com responsável, data e histórico.
2	Calcular sugestão de compra a partir do estoque mínimo e permitir ajuste manual antes da cotação.
3	Completar a conferência de recebimento com divergências detalhadas, nota fiscal e ação tomada.
4	Conectar as operações de compra, entrada, movimentação e atualização de estoque em uma transação consistente.
5	Criar consultas para histórico, auditoria e indicadores de compras e perdas.
6	Revisar a configuração de segurança da API e documentar o processo de criação/gestão de contas.

Conclusão

O repositório apresenta a base de um sistema de compras e estoque com interface React, API REST em ASP.NET Core e persistência via Entity Framework Core/PostgreSQL . Os cadastros, cotações, pedidos e operações de estoque formam o núcleo funcional. O documento atualizado registra essa implementação e deixa explícitos os itens ainda necessários para cobrir todo o processo descrito pela equipe.